PIAZ LANG

A Domain-Specific Language & Laboratory Environment
for Advanced Networking, Systems, Binary Work & Authorized Security Engineering

Internal Technical Proposal — Expanded Edition
Prepared for Company Red Team & Infrastructure Engineering

Perspective: 30-year IT & Network Engineering Practitioner
Version 1.1 | 2026

Packets • Flows • Tunnels • Custom Protocols • P2P • eBPF • Kernel
Memory & Hex • Assembly • Bit-level Ops • Graph / State-machine Engine
Hashing & Custom Crypto • JIT / AOT • Channels & Concurrency

1. Executive Summary

After three decades designing, operating, attacking (under authorization), and defending networks, the tooling landscape remains fragmented. Socket work, packet crafting, eBPF, kernel modules, tunneling, binary inspection, custom protocols, concurrency, and cryptographic primitives live in separate languages and toolchains.

Piaz Lang is a single purpose-built language and laboratory whose core vocabulary is networks, flows, packets, tunnels, kernel hooks, memory views, graphs, and cryptographic building blocks. It is designed so that an engineer can move from idea to running, observable, auditable artifact with minimal context switching — while every sensitive capability remains under explicit authorization and structured audit.

2. The Problem — Practitioner View

A senior engineer currently maintains muscle memory across Python/Scapy, Go/Rust, Bash+socat, C+libbpf, kernel toolchains, Ghidra/IDA + hex editors, and assorted one-offs. Context switching is expensive; the same tunnel or protocol idea is re-implemented multiple times; juniors lack a coherent mental model; observability and audit trails are inconsistent.

3. Vision & Design Principles

- **Network-native** — Endpoints, flows, tunnels, routes, interfaces are first-class values.
- **Protocol-extensible** — New socket protocols and codecs can be defined inside the language.
- **P2P & distributed** — Instances talk to each other and to foreign code over built-in channels.
- **Low-level accessible** — Controlled memory views, hex, assembly, OS APIs, bit operations.
- **Graph-centric** — Built-in computational graphs, state machines, workflow and rule engines.
- **Crypto-native** — Primitives for keypairs, HMAC, and constructing new protocols.
- **Concurrent** — Channels, threads, locks, atomics, synchronization as language features.
- **Compiled** — Both JIT for interactive work and AOT for production binaries.
- **Observable & auditable** — Telemetry and authorization context by default.
- **Lab-oriented** — Reproducible multi-node topologies and scenario runners.

4. Core Networking & Tunneling (recap)

Local / remote / dynamic / multi-hop forwarding, packet & flow values, rewrite pipelines, and first-class path composition remain foundational. Examples:

```
:8080 => 10.0.0.5:80 via ssh://user@jump.example
remote :9000 => localhost:3000 via ssh://user@jump.example
path = ssh://u1@b1 -> ssh://u2@b2 -> ssh://u3@target
:8080 => internal.svc:80 using path
flow = capture iface eth0 filter "tcp port 80"
flow |> rewrite ttl 64 |> encapsulate geneve vni 42 |> sendto 10.0.0.2
```

5. Extended Core Capabilities (with Syntax)

5.1 P2P Communication Between Instances

Any running Piaz instance can discover and talk to other instances (or foreign processes that speak the Piaz wire protocol). Useful for distributed labs, multi-host red-team scenarios under authorization, and coordinating tools.

```
// Advertise this instance
p2p.advertise(name: "lab-node-3", port: 7700)

// Connect to another instance
peer = p2p.connect("lab-node-7:7700")
peer.send(msg{type: "status", payload: metrics})
reply = peer.recv(timeout: 2s)

// Broadcast to a group
p2p.group("red-lab").broadcast(event{action: "start_capture"})
```

5.2 Creating New Socket / Wire Protocols

Define a new protocol as a first-class value: framing, handshake, message types, and state machine. The runtime can then open sockets that speak it.

```
protocol ShadowTalk {
version = 1
handshake = { send magic 0xSHDW; recv ack; exchange nonces }
frame = length-prefixed(u32) + payload
messages = {
Ping { id: u64 }
```

```
Pong { id: u64, rtt: u32 }
Data { stream: u16, flags: u8, body: bytes }
}
}

sock = listen :9900 protocol ShadowTalk
conn = sock.accept()
conn.send(Data{stream: 1, flags: 0, body: payload})
```

5.3 Importing Code & One-Round-Trip Remote Calls

Local modules can be imported normally. In addition, a remote Piaz instance can expose functions that are callable in a single round-trip (inspired by Redis + Lua), useful for keeping logic next to data or next to a privileged vantage point.

```
// Local import
import "lab/scanners.piaz" as scan
hosts = scan.arp_sweep("10.0.0.0/24")

// Expose a function on this instance for remote callers
export fn collect_flows(iface: str, secs: int) -> FlowSet {
return capture(iface).take(secs).collect()
}

// Call it from another instance in one round-trip
peer = p2p.connect("sensor-1:7700")
flows = peer.call("collect_flows", "eth0", 5)
// Equivalent short form:
flows = p2p://sensor-1/collect_flows("eth0", 5)
```

5.4 Channels, Threads, Locking, Sync, Atomics

Lightweight threads (or tasks), typed channels, mutexes, rwlocks, condition variables, and atomic primitives are part of the language core so concurrent network and analysis code stays readable.

```
ch = channel<Packet>(capacity: 1024)

go { // spawn
for p in capture("eth0") { ch.send(p) }
}

go {
for p in ch {
process(p)
}
}

counter = atomic<u64>(0)
counter.fetch_add(1)

mu = mutex()
mu.lock() { shared_map.insert(k, v) } // scoped lock
```

5.5 JIT and AOT Compilation

Interactive sessions and short-lived lab tools use JIT for fast startup and dynamic specialization. Production helpers and long-running services can be AOT-compiled to native binaries.

```
// JIT (default in REPL / script)
piaz run lab_scenario.piaz

// AOT to native binary
piaz build --aot --opt 2 lab_scenario.piaz -o scenario

// Explicit hot-path JIT annotation
#[jit]
fn fast_filter(p: Packet) -> bool { p.tcp.dst == 443 && p.len > 100 }
```

5.6 Memory Dumping & OS-Level API Calls

Controlled inspection of process memory (own process or, under authorization, target lab processes) and direct but gated access to OS APIs. Intended for lab instrumentation, custom protocol stacks, and authorized analysis — always under audit.

```
// Dump a region of current process
mem = memory.dump(addr: 0x7fabc000, len: 4096)
hexprint(mem)

// Open a view (read-only by default)
view = memory.view(pid: lab_pid, addr: 0x400000, len: 0x1000, mode: "r")
```

```
bytes = view.read(offset: 0x120, n: 32)

// OS API (gated)
fd = os.open("/proc/net/tcp", "r")
data = os.read(fd, 8192)
os.close(fd)

// Raw socket example
rs = os.socket(AF_PACKET, SOCK_RAW, ETH_P_ALL)
```

5.7 Hex-Level Inspection & Controlled Binary Patching

First-class hex views and pattern search over binary blobs or memory regions. Controlled patching is supported for **lab binaries and artifacts under authorization** (e.g. instrumenting a test binary, fixing a fixture, or implementing a custom loader). The runtime records every mutation for audit. This is not a general-purpose cracking toolkit.

```
bin = file.read_bytes("lab_target.bin")
hx = hex.view(bin)

// Search
hits = hx.find(pattern: "48 89 e5 41 57") // common prologue
hits = hx.find_string("config.json")

// Controlled patch (lab artifact only, audited)
hx.patch(offset: 0x2a30, bytes: "90 90 90 90") // NOP sled example
file.write_bytes("lab_target_patched.bin", hx.bytes())

// Disassemble a slice (integration point)
insns = asm.disasm(hx.slice(0x1000, 64), arch: "x86_64")
```

5.8 Assembly Support

Inline assembly and standalone assembly modules for the platforms the lab targets, with clear boundaries between safe Piaz code and raw instructions.

```
fn read_tsc() -> u64 {
asm {
rdtsc
shl rdx, 32
or rax, rdx
// returns rax
}
}

// Assemble a small stub
stub = asm.assemble("
mov rax, 1
ret
", arch: "x86_64")
memory.write(lab_addr, stub)
```

5.9 Codecs & Data Hiding / Compression Primitives

Building blocks for custom codecs and for embedding structured data inside cover traffic or noise (useful for covert-channel research in authorized exercises and for resilient protocol design).

```
codec = codec.define {
encode(msg) {
noise = random_bytes(len(msg) * 4)
return interleave(msg, noise) |> xor_keystream(session_key)
}
decode(blob) {
return deinterleave(xor_keystream(blob, session_key))
}
}

wire = codec.encode(real_payload)
recovered = codec.decode(wire)
```

5.10 Bit-Level Operations

Ergonomic bit fields, masks, shifts, rotates, population count, and bit-level views of packets or buffers.

```
flags = u16(0b0001_0100_1100_0011)
flags.set_bit(7, true)
flags.clear_bit(1)
flags.toggle_bit(3)
n = flags.popcount()
```

```
// Bit-field view of a header
hdr = bitview(packet[0..4])
version = hdr[0..4] // bits 0-3
hdr[4..8] = 0x5 // write IHL

rotated = flags.rotate_left(3)
masked = flags & 0x0FF0
```

5.11 Built-in Graph & State-Machine Engine

A first-class graph engine supports computational graphs, state machines, workflow engines, rule engines, and report pipelines. Most common graph algorithms are available out of the box.

```
// State machine for a protocol
sm = state_machine {
states: [Idle, Handshaking, Ready, Closing]
start: Idle
transitions: {
Idle --syn--> Handshaking
Handshaking --ack--> Ready
Ready --fin--> Closing
Closing --done--> Idle
}
}
sm.on(Ready) { start_data_plane() }

// Computational / workflow graph
g = graph()
g.add_node("capture", capture_fn)
g.add_node("filter", filter_fn)
g.add_node("enrich", enrich_fn)
g.add_node("report", report_fn)
g.edge("capture", "filter")
g.edge("filter", "enrich")
g.edge("enrich", "report")
g.run(input: iface_name)

// Classic algorithms
path = g.shortest_path("A", "Z", weight: "latency") // Dijkstra / A*
order = g.topo_sort()
comps = g.strongly_connected()
tour = g.tsp_approx("start", cities) // traveling
flow = g.max_flow("source", "sink")
match = g.bipartite_matching()
centrality = g.betweenness()
```

5.12 Hashing, Keypairs, HMAC & Custom Encryption Protocols

Low-level cryptographic building blocks so new protocols can be constructed and analyzed inside the lab. Includes hash functions, keypair generation, HMAC, symmetric primitives, and a way to compose them into named protocols with clear state machines.

```
// Hashing
h = hash.sha256(data)
h = hash.blake3(data)
h = hash.custom(sponge_params, data) // experimental constructions

// Keypairs
kp = crypto.keypair.generate("x25519")
kp = crypto.keypair.generate("ed25519")
sig = kp.sign(message)
ok = kp.public.verify(message, sig)

// HMAC
tag = crypto.hmac(key, data, hash: "sha256")

// Custom protocol sketch
protocol SecureLab {
handshake = {
ephemeral = keypair.generate("x25519")
send ephemeral.public
peer_pub = recv()
shared = ephemeral.dh(peer_pub)
session_key = kdf(shared, "lab-v1")
}
```

```
seal(msg) = aead_encrypt(session_key, msg)
unseal(blob)= aead_decrypt(session_key, blob)
}

conn = dial("peer:9900", protocol: SecureLab)
conn.send(seal(payload))
```

6. High-Level Architecture

Layer	Responsibility
Surface Language	Syntax, modules, P2P/export, graph & state-machine DSL, protocol definitions
Compiler Frontend	Parse, type-check, lower to IR; JIT specialization hooks
IR + Backends	JIT (interactive) and AOT (native) backends; optional eBPF lowering
Runtime	Scheduler, channels, atomics, tunnels, P2P fabric, telemetry, audit
Systems / Memory	Raw sockets, OS APIs (gated), memory views, hex services, asm helpers
Graph Engine	Computational graphs, state machines, classic graph algorithms
Crypto Toolbox	Hashes, keypairs, HMAC, AEAD, KDF, protocol composition
Lab Orchestrator	Namespaces, topologies, scenario runners, artifact management
Audit & Policy	Authorization context, mandatory logging, capability gating

Recommended implementation host: Rust for the runtime and compilers, reusing mature eBPF, crypto, and packet crates where they do not break the unified model.

7. Security, Ethics & Compliance Posture

Powerful capabilities (memory views, binary mutation, custom protocols, P2P, OS APIs) are dual-use. The design therefore encodes a strong bias toward authorized, auditable operation:

- Sensitive operations require an explicit authorization context object.
- Every tunnel, memory view, binary patch, kernel attachment, and remote call emits structured audit events.
- Binary mutation and cross-process memory access are restricted to designated lab artifacts and targets.
- Documentation and examples focus on defensive validation, authorized red-team exercises, protocol research, and infrastructure automation.
- Distribution targets internal company use; the most powerful capabilities are not enabled in unrestricted public builds.

Piaz Lang is an engineering instrument for teams that already operate under legal and organizational authority. It is not positioned as a general cracking or unauthorized-access toolkit.

8. Suggested Roadmap

Phase	Focus	Outcomes
0 — Foundation	Core language, runtime, sockets, flows, channels, atomics	Runnable scripts with concurrency and basic networking
1 — Tunnels + P2P	Local/remote/dynamic/multi-hop, P2P fabric, remote call	Replace most ad-hoc tunneling and glue scripts
2 — Protocols + Packets	Custom protocol definitions, rich packet model, codecs	New wire protocols expressible inside the language
3 — eBPF + Memory + Asm	eBPF bridge, memory views, hex services, assembly	Low-level lab work without leaving the environment
4 — Graph + Crypto	Graph/state-machine engine, hashing, keypairs, custom crypto protocols	Workflow, rule, and crypto-protocol construction in one place
5 — JIT/AOT + Hardening	Production AOT, policy engine, full audit coverage, packaging	Internal production-ready platform

9. Why Build This Internally

- Shared vocabulary across red team, infrastructure, and platform engineers.
- Faster iteration on custom network behaviors, protocols, and authorized scenarios.
- One environment for tunnels, P2P, packets, eBPF, memory, graphs, and crypto construction.
- Policy and audit enforced at the language/runtime level rather than by convention.
- Strategic asset that compounds as more internal tooling is expressed in it.

10. Closing Recommendation

Approve a focused Phase 0/1 spike: core language + runtime + channels + basic tunnels + P2P. Success metric: number of existing scripts and manual procedures that can be retired in favor of short, readable, audited Piaz programs. Subsequent phases add protocols, low-level services, graph engine, and crypto toolbox once the foundation is stable.

Document status: Internal proposal — not for external distribution.
Piaz Lang concept, syntax sketches, and capability list are company confidential.